System Design Document: AI-Powered Real Estate Property Valuation Assistant

Author / Presenter: Kunjal Ahuja

Domain: Real Estate & PropTech Valuation Intelligence

Capstone: Prodapt AI/ML Engineering Capstone

1. Executive Summary & Problem Statement

Real estate professionals, underwriters, and appraisers regularly perform recurring workflows involving comparative market analysis (CMA), physical condition and age adjustments, and market volatility risk assessments. Historical property databases contain thousands of transactional records with complex physical, geographic, and economic features. However, traditional appraisal relies on manual searching across static MLS records, subjective condition adjustments, and unquantified risk adjustments.

This system delivers an end-to-end autonomous, multi-agent valuation platform and user interface that enables real estate agents to enter structured property parameters or natural language queries to:

1. Retrieve prioritized historical comparable sales using true hybrid search (dense cosine embeddings + lexical TF-IDF keyword scoring) with automated neighborhood widening.
2. Predict fair market value using a supervised Gradient Boosting Regressor (R^2 = 0.9198, RMSE = \25,361.84$, MAPE = 8.87\%) with 90% empirical prediction intervals.
3. Assess structural, condition, liquidity, and market volatility risks via a composite 0–100 index.
4. Dynamically reconcile valuation signals using a `comp_fit` weighting formula.
5. Audit valuations through an independent LLM-as-Judge appraisal auditor.
6. Continuously adjust for systematic pricing bias via an uncalibrated feedback loop.

2. High-Level Architecture & End-to-End Data Flow

Code:

```
[Ames Housing Dataset (10,000 Records: 2,930 Real + 7,070 Synthetic)]
■
▼
[Data Pipeline & Standardizer]
(15 Standardized Key Fields, 1-10 Quality Scale)
■
■■■
```

3. Technology Selection & Architectural Trade-offs

Component	Selected Technology	Alternative Considered	Rationale & Architectural Trade-offs
Programming Language	Python 3.10+ (tested on Python 3.13)	Go / TypeScript	Native ecosystem support for Scikit-learn, PyTorch, Sentence-Transformers, DeepEval, and FastAPI.
Backend API	FastAPI + Uvicorn	Flask / Django	Asynchronous high-throughput, native Pydantic validation, and automatic OpenAPI schema generation.
Frontend UI	React 18.3.1 + Vite 5.4.11	Streamlit	Production-grade single-page application with responsive layout, accessible tab navigation (7 tabs), persistent state, dark mode, and zero page reloads. Streamlit retained as legacy prototype only.
Embedding Model	`all-MiniLM-L6-v2`	OpenAI `text-embedding-3-small`	384-dimensional dense vectors with local sub-millisecond inference; zero recurring API expenses or network latency.
Vector Store	In-Memory NumPy Cosine Dot-Product Index	Pinecone / Milvus / Chroma	With 10,000 records, an in-memory normalized NumPy matrix (`embeddings.npy`, 15.3 MB) provides sub-5ms cosine retrieval with zero database server overhead or external network calls.
Lexical Keyword Search	Scikit-learn `TfidfVectorizer`	Elasticsearch / Meilisearch	Embedded directly in the application runtime; evaluates full candidate sets without hosting separate search clusters.
Supervised ML Regressor	`GradientBoostingRegressor`	Random Forest / MLP	Outperformed baseline Random Forest on real test evaluation (R^2 = 0.9198 vs. 0.9031), offering lower RMSE (\$25,361.84 vs. \$27,870.39) and robust residual-based prediction intervals.
LLM & Reasoning	Dual-Mode LLM (`gpt-4o-mini` with Deterministic Fallbacks)	Cloud-Only LLM	Guarantees all core valuation and audit endpoints operate completely offline without failing when API keys are absent.
Evaluation Framework	DeepEval (v4.2.8 pinned)	RAGAS / Manual Review	Native support for LLMTestCase, Faithfulness, Relevancy, Precision, and Recall. Clarification: RAGAS was evaluated but DeepEval was selected for its robust local orchestration and test case abstractions.

4. Key Assumptions & Interpretations

1. Dataset Structure & Synthetic Augmentation:

- The underlying dataset contains 10,000 records: 2,930 real sales (Dean De Cock, 2011; OpenML 43926) and 7,070 synthetic sales generated via Gaussian Copula with empirical marginals and novelty distance filtering.
- **Honest Test Split**: The test set (n = 586) strictly contains 20% of the real records (`random_state=42`). Synthetic records are used only in training (n = 9,414) and are never present in the test set.
- **Comparable Isolation**: Comparable property searches exclude synthetic rows by default (`include_synthetic=False`) so appraisers and clients see only verified historical transactions.

2. Dynamic Comparable Search Widening:

- When a subject property has fewer than 3 comps in its target neighborhood that match within +/- 25\% living area and +/- 1 quality grade, the search automatically widens city-wide (relaxing to +/- 40\% and +/- 2 if needed) to ensure representative comps.

3. Dynamic Valuation Reconciliation (`comp_fit`):

- Rather than an arbitrary 50/50 split, the system dynamically weights the comparable median price and the ML statistical estimate based on `comp_fit`:

{comp_fit} = {{matching comps}}{{comps found}} x \min<=ft(1, {{comps found}}{5}\right)

{comp_weight} = {clip}(0.5 x {comp_fit} + 0.1, 0.1, 0.5)

{ml_weight} = 1.0 - {comp_weight}

- When comp evidence is strong and closely matching, comp weight reaches 50%. When local comps are sparse or divergent, ML weight increases up to 90%.

4. Feedback Calibration Drift Prevention:

- Successive feedback submissions evaluate error and bias against uncalibrated estimates (recommended / applied_factor), preventing compounding feedback loops from causing calibration drift.

5. Strict Filter Handling:

- If user-provided metadata filters match zero properties, the retriever returns an empty list (`[]`) rather than falling back to unconstrained properties. Downstream valuation gracefully defaults to ML prediction.

---

5. Retrieval Strategy & Comparable Prioritization

5.1 Searchable Document Construction

Each property row is converted into a rich textual document without splitting (1 property = 1 document):

Code:
```
Property PROP-0001 located in North Ames neighborhood. Living area: 1656 sq ft, Lot area: 31770 sq ft.
Accommodations: 3 bedrooms, 1 full bathrooms, 7 total rooms above ground. Basement: 1080 sq ft. Garage: 2 cars
(528 sq ft). Built in 1960, remodeled in 1960. Overall quality rating: 6/10, overall condition rating: 5/10.
Historical sale price: $215,000 ($129.83/sq ft).
```

5.2 True Hybrid Fusion Formula

Dense semantic cosine scores and TF-IDF lexical scores are computed over the full candidate pool and fused via convex combination:

{Score}_{{hybrid}} = alpha \cdot {Score}_{{semantic}} + (1 - alpha) \cdot {Score}_{{keyword}}

where alpha = 0.55 (55% dense semantic similarity, 45% lexical keyword similarity).

5.3 Comparable Prioritization Formula

Candidates from the hybrid retrieval pool are ranked by a composite score blending hybrid similarity and physical property proximity:

{Score}_{{prioritized}} = 0.50 \cdot {Score}_{{hybrid}} + 0.50 \cdot {Score}_{{comparability}}

Where physical comparability is computed across 5 weighted dimensions:

- **Location Proximity (30%)**: 1.0 for same neighborhood, 0.8 for adjacent, 0.35 for other.
- **Living Area Proximity (25%)**: $1.0 - \min(|{SqFt}_{{comp}} - {SqFt}_{{target}}| / \max({SqFt}_{{target}}, 500), 1.0)$
- **Overall Quality Proximity (20%)**: $1.0 - |{Qual}_{{comp}} - {Qual}_{{target}}| / 9.0$
- **Age / Era Proximity (15%)**: $1.0 - |{Year}_{{comp}} - {Year}_{{target}}| / 50.0$
- **Room Count Proximity (10%)**: Delta penalty based on total bedroom and bathroom differences.

---

6. Multi-Agent System Architecture

1. **Guardrail Agent**: Enforces strict physical parameter constraints (14 fields: living area 100–15,000 sq ft, quality 1–10, etc.) and detects prompt injection attacks. Rejects invalid requests with HTTP 400.
2. **Classification Agent**: Segments properties into Market Tiers (Luxury, Upper-Middle, Mid-Market, Affordable), Typologies (Executive Estate, Suburban Single-Family, Townhome, Starter), and Preservation Grades (Grade A to D).
3. **Valuation Agent**: Computes comparable price statistics (median, mean, size-adjusted PPSF fair value) and produces a plain-English explanation of valuation rationale.
4. **Predictive ML Regressor**: Generates statistical point estimates and 90% empirical prediction intervals using the trained `GradientBoostingRegressor`.
5. **Property Value Risk Agent**: Scores risk (0–100) across physical depreciation (30%), quality-condition gap (25%), neighborhood price volatility (25%), and layout liquidity (20%).
6. **Feature & Location Impact Agent**: Analyzes neighborhood price premiums relative to city baselines and performs what-if renovation simulations.
7. **Synthesis & Calibration Engine**: Applies dynamic `comp_fit` weighting between comps and ML prediction, incorporates feedback calibration factors, and scales prediction intervals.
8. **LLM-as-Judge Auditor**: Audits reconciled values against comp medians and ML bounds, producing structured verdicts (`APPROVED`, `FLAGGED_OVERVALUED`, `FLAGGED_UNDERVALUED`, `NEEDS_REVIEW`) and plain-English recommendations.

---

7. Continuous Learning & Feedback Loop

The platform includes a persistent feedback store (`data/sample_data/feedback.json`) that logs verified transactions:

1. Appraisers submit verified sales prices, star ratings, and optional notes.
2. The service computes Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE).
3. Systematic pricing bias is evaluated against uncalibrated estimates:

{Uncalibrated} = {{Recommended}}{{AppliedCalibrationFactor}}

{Bias}_{\%} = {1}{N} \sum_{i=1}^N {{Actual}_i - {Uncalibrated}_i}{{Uncalibrated}_i} x 100

4. The calibration factor is computed as:

{CalibrationFactor} = 1.0 + {{Bias}_{\%}}{100}

5. The orchestrator applies this factor directly during valuation reconciliation to eliminate systematic drift.

---

8. Evaluation Framework (DeepEval Quality Benchmark)

DeepEval (v4.2.8) is integrated into `src/services/evaluation_service.py` with 5 benchmark scenarios (`EVAL-01` through `EVAL-05`) evaluating end-to-end quality:

- **FaithfulnessMetric**: Assesses whether generated narratives align factually with retrieved comps.
- **AnswerRelevancyMetric**: Assesses whether generated outputs answer the specific valuation query.
- **ContextualPrecisionMetric**: Assesses whether the most relevant historical comps are ranked highest.
- **ContextualRecallMetric**: Assesses whether all critical valuation dimensions are covered.
- **Valuation Accuracy**: Measures whether reconciled FMV falls within historical ground-truth bounds.

Honest Dual-Mode Operation:

- In **offline mode** (no API key configured), DeepEval returns `evaluation_mode = "offline_no_llm"` and marks LLM-backed metrics as skipped (`SKIPPED_NO_LLM_KEY`), while deterministically computing valuation accuracy.
- In **live mode** (`OPENAI_API_KEY` configured), real metric calculations execute via the configured LLM, returning measured scores (100% pass rate on live test run).

---

9. Frontend Architecture (React 18 + Vite)

The user interface (`frontend/`) is a modular, high-performance React 18 single-page application:

- **Sidebar Form**: Shared property input form (28 Ames neighborhoods, living area, lot area, bedrooms, full baths, year built, basement sq ft, garage cars, condition 1–10, quality 1–10, optional notes).
- **7 Dedicated Tabs**:

1. *Valuation*: Headline reconciled price, 90% confidence range, clean range bar, 2-sentence plain-English explanation, and quick audit button.
2. *Comparable sales*: Real historical transactions table, similarity scores, and physical comparability details.
3. *ML prediction*: Gradient Boosting point estimate, 90% prediction interval, and top feature drivers.
4. *What-if*: Interactive renovation simulator (room additions, finish upgrades, garage expansions).
5. *Risk and audit*: Multi-factor risk breakdown (0–100 score) and LLM judge audit verdict.
6. *Feedback*: Appraiser rating submission and live bias calibration monitoring.
7. *Evaluation*: Live DeepEval benchmark runner and scenario breakdown.

- **UX & Accessibility**: Dark/light theme toggle with persistent storage, accessible keyboard navigation (`aria-selected`), and out-of-date indicators when form parameters change.